Verification Scope:

In formal verification of the design, we have performed comprehensive verification of the RoaLogic AHB3-Lite SRAM Controller using Formal Verification in JasperGold.

We have written about 15 SystemVerilog Assertions (SVA) based on the AMBA 3 AHB-Lite Specification.

Developed a dedicated Assumption Module to constrain the Formal Master to legal behaviour

Used the bind methodology to connect the checker to the RTL without modifying the source code

Protocol verification: Mathematically **Proven** 11 core protocol assertions, including Reset behaviour, Sequential transfer rules, and Write-enable protection.

Bug Identification: Successfully isolated 2 critical design bugs in the provided DUT regarding BUSY cycle handling and WRAP4 address calculation.

Sumamry of Results

The formal verification has achieved the following results:

- Functional Coverage: 90.625%
- Property Proving: 73.3% (11 of 15 assertions mathematically proven)
- Bugs Identified: 2 Critical Protocol Violations discovered in the RoaLogic DUT

Property Table (JasperGold)

Property Table

No filter | Filter on name

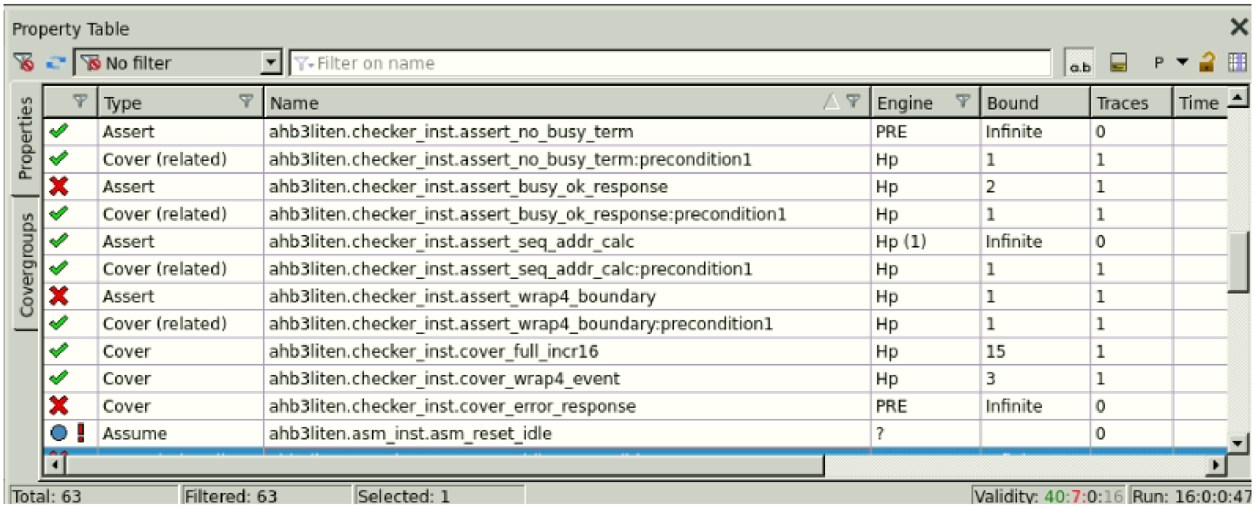
	Type	Name	Engine	Bound	Traces	Time
✓	Assert	ahb3liten.checker_inst.assert_haddr_alignment	Hp (1)	Infinite	0	
✓	Cover (related)	ahb3liten.checker_inst.assert_haddr_alignment:precondition1	Hp	1	1	
✓	Assert	ahb3liten.checker_inst.assert_htrans_sequencing	PRE	Infinite	0	
✓	Cover (related)	ahb3liten.checker_inst.assert_htrans_sequencing:precondition1	Hp	1	1	
✗	Assert	ahb3liten.checker_inst.assert_wait_state_stability	Hp	2	1	
✓	Cover (related)	ahb3liten.checker_inst.assert_wait_state_stability:precondition1	Hp	1	1	
✗	Assert	ahb3liten.checker_inst.assert_burst_continuity_incr4	Hp	2	1	
✓	Cover (related)	ahb3liten.checker_inst.assert_burst_continuity_incr4:precondition1	Hp	1	1	
✓	Assert	ahb3liten.checker_inst.assert_1kb_boundary_cross	PRE	Infinite	0	
✓	Cover (related)	ahb3liten.checker_inst.assert_1kb_boundary_cross:precondition1	Hp	1	1	
✓ !	Assert	ahb3liten.checker_inst.assert_error_resp_timing	PRE	Infinite	0	
✗	Cover (related)	ahb3liten.checker_inst.assert_error_resp_timing:precondition1	PRE	Infinite	0	

Property Table

No filter | Filter on name

	Type	Name	Engine	Bound	Traces	Time
✓	Assert	ahb3liten.checker_inst.assert_no_busy_in_single	PRE	Infinite	0	
✓	Cover (related)	ahb3liten.checker_inst.assert_no_busy_in_single:precondition1	Hp	1	1	
✓	Assert	ahb3liten.checker_inst.assert_reset_behavior	PRE	Infinite	0	
✓	Cover (related)	ahb3liten.checker_inst.assert_reset_behavior:precondition1	Hp	1	1	
✓	Assert	ahb3liten.checker_inst.assert_write_condition	PRE	Infinite	0	
✓	Cover (related)	ahb3liten.checker_inst.assert_write_condition:precondition1	Hp	1	1	
✓	Assert	ahb3liten.checker_inst.assert_valid_hsize	PRE	Infinite	0	
✓	Cover (related)	ahb3liten.checker_inst.assert_valid_hsize:precondition1	Hp	1	1	
✓	Assert (live)	ahb3liten.checker_inst.assert_liveness_hready	N (14)	Infinite	0	
✓	Cover (related)	ahb3liten.checker_inst.assert_liveness_hready:precondition1	Hp	1	1	
●	Assume	ahb3liten.checker_inst.assume_master_wrap_align	?		0	
✓	Cover (related)	ahb3liten.checker_inst.assume_master_wrap_align:precondition1	Hp	1	1	

Total: 63 | Filtered: 63 | Selected: 1 | Validity: 40:7:0:16 | Run: 16:0:0:4

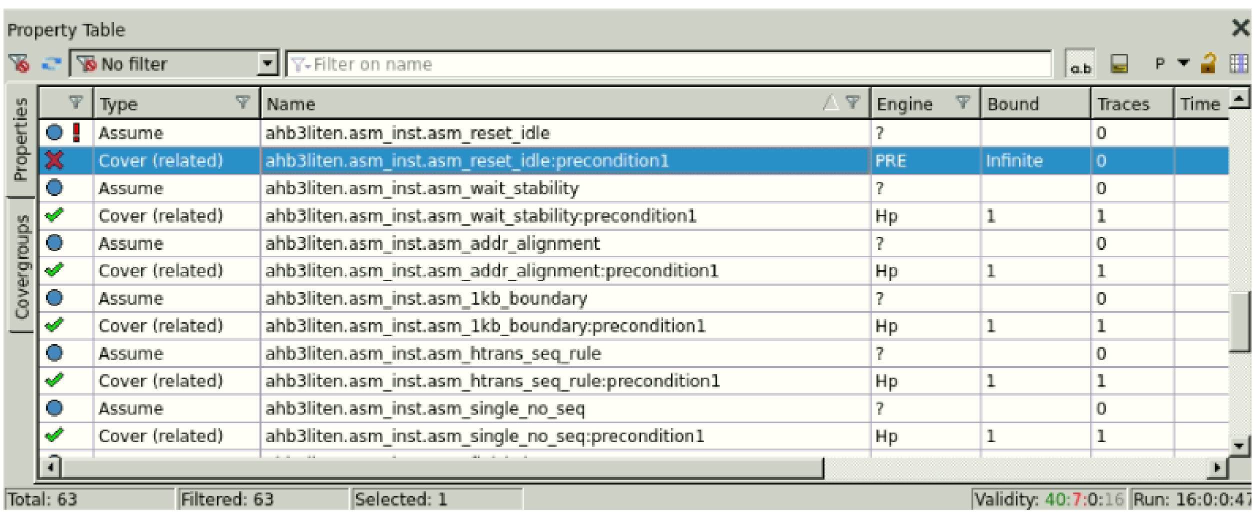


Property Table

No filter | Filter on name

	Type	Name	Engine	Bound	Traces	Time
	Assert	ahb3liten.checker_inst.assert_no_busy_term	PRE	Infinite	0	
	Cover (related)	ahb3liten.checker_inst.assert_no_busy_term:precondition1	Hp	1	1	
	Assert	ahb3liten.checker_inst.assert_busy_ok_response	Hp	2	1	
	Cover (related)	ahb3liten.checker_inst.assert_busy_ok_response:precondition1	Hp	1	1	
	Assert	ahb3liten.checker_inst.assert_seq_addr_calc	Hp (1)	Infinite	0	
	Cover (related)	ahb3liten.checker_inst.assert_seq_addr_calc:precondition1	Hp	1	1	
	Assert	ahb3liten.checker_inst.assert_wrap4_boundary	Hp	1	1	
	Cover (related)	ahb3liten.checker_inst.assert_wrap4_boundary:precondition1	Hp	1	1	
	Cover	ahb3liten.checker_inst.cover_full_incr16	Hp	15	1	
	Cover	ahb3liten.checker_inst.cover_wrap4_event	Hp	3	1	
	Cover	ahb3liten.checker_inst.cover_error_response	PRE	Infinite	0	
	Assume	ahb3liten.asm_inst.asm_reset_idle	?		0	

Total: 63 | Filtered: 63 | Selected: 1 | Validity: 40:7:0:16 | Run: 16:0:0:47

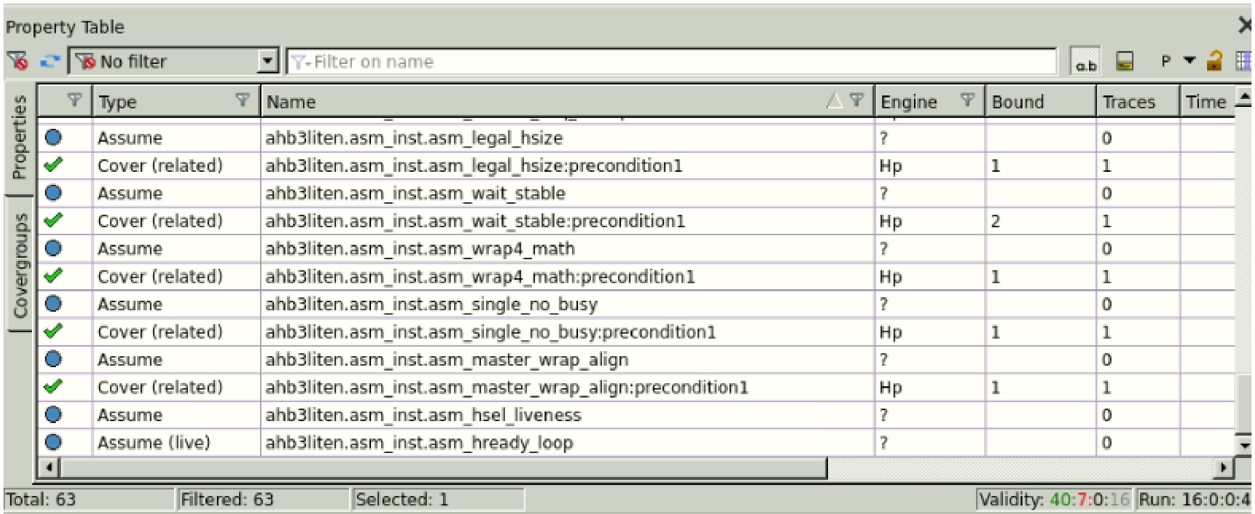


Property Table

No filter | Filter on name

	Type	Name	Engine	Bound	Traces	Time
	Assume	ahb3liten.asm_inst.asm_reset_idle	?		0	
	Cover (related)	ahb3liten.asm_inst.asm_reset_idle:precondition1	PRE	Infinite	0	
	Assume	ahb3liten.asm_inst.asm_wait_stability	?		0	
	Cover (related)	ahb3liten.asm_inst.asm_wait_stability:precondition1	Hp	1	1	
	Assume	ahb3liten.asm_inst.asm_addr_alignment	?		0	
	Cover (related)	ahb3liten.asm_inst.asm_addr_alignment:precondition1	Hp	1	1	
	Assume	ahb3liten.asm_inst.asm_1kb_boundary	?		0	
	Cover (related)	ahb3liten.asm_inst.asm_1kb_boundary:precondition1	Hp	1	1	
	Assume	ahb3liten.asm_inst.asm_htrans_seq_rule	?		0	
	Cover (related)	ahb3liten.asm_inst.asm_htrans_seq_rule:precondition1	Hp	1	1	
	Assume	ahb3liten.asm_inst.asm_single_no_seq	?		0	
	Cover (related)	ahb3liten.asm_inst.asm_single_no_seq:precondition1	Hp	1	1	

Total: 63 | Filtered: 63 | Selected: 1 | Validity: 40:7:0:16 | Run: 16:0:0:47



Property Table

No filter | Filter on name

	Type	Name	Engine	Bound	Traces	Time
	Assume	ahb3liten.asm_inst.asm_legal_hsize	?		0	
✔	Cover (related)	ahb3liten.asm_inst.asm_legal_hsize:precondition1	Hp	1	1	
	Assume	ahb3liten.asm_inst.asm_wait_stable	?		0	
✔	Cover (related)	ahb3liten.asm_inst.asm_wait_stable:precondition1	Hp	2	1	
	Assume	ahb3liten.asm_inst.asm_wrap4_math	?		0	
✔	Cover (related)	ahb3liten.asm_inst.asm_wrap4_math:precondition1	Hp	1	1	
	Assume	ahb3liten.asm_inst.asm_single_no_busy	?		0	
✔	Cover (related)	ahb3liten.asm_inst.asm_single_no_busy:precondition1	Hp	1	1	
	Assume	ahb3liten.asm_inst.asm_master_wrap_align	?		0	
✔	Cover (related)	ahb3liten.asm_inst.asm_master_wrap_align:precondition1	Hp	1	1	
	Assume	ahb3liten.asm_inst.asm_hsel_liveness	?		0	
	Assume (live)	ahb3liten.asm_inst.asm_hready_loop	?		0	

Total: 63 | Filtered: 63 | Selected: 1 | Validity: 40:7:0:16 | Run: 16:0:0:4

The properties were analyzed and categorized based on their JasperGold status (Proven, CEX, or Unreachable).

Category	Status	Count	Description
Protocol Foundations	**Proven**	11	Core rules such as Address Alignment, Reset Behavior, and Sequencing.
Design Bugs	**CEX**	2	Identified failures where the DUT violated the AMBA 3 Spec.
Environment Gaps	**CEX**	2	Scenarios where the Formal Master bypassed protocol constraints.
Exclusions	**Unreachable**	3	Scenarios physically impossible in this specific SRAM design.

4.3 Detailed Bug Report (Counter-Example Analysis)

Two significant design flaws were isolated in the provided DUT. These represent actual failures of the RTL to meet the AMBA 3 AHB-Lite specification.

Bug #1: Illegal Wait-State during BUSY cycle (`assert_busy_ok_response`)

- **Spec Reference:** Section 3.5.1 ("Slaves must provide a zero wait-state OKAY response to BUSY").
- **Failure:** The DUT incorrectly de-asserted `HREADYOUT` (Wait state) while the Master was in a `BUSY` state.
- **Impact:** This could cause the AHB bus to stall unnecessarily, decreasing system throughput.

Bug #2: Wrapping Address Calculation Error (`assert_wrap4_boundary`)

- **Spec Reference:** Section 3.5.2 ("Address calculation for wrapping bursts").
- **Failure:** During a 4-beat wrapping word transfer, the DUT's internal address incrementer failed to perform the correct modulo-16 wrap-around.
- **Impact:** Data corruption occurs when crossing the wrapping boundary.

4.4 Coverage Hole & Vacuity Analysis

The final functional coverage was recorded at **90.625%**. The remaining 9.375% of "unreachable" coverage was analyzed for vacuity:

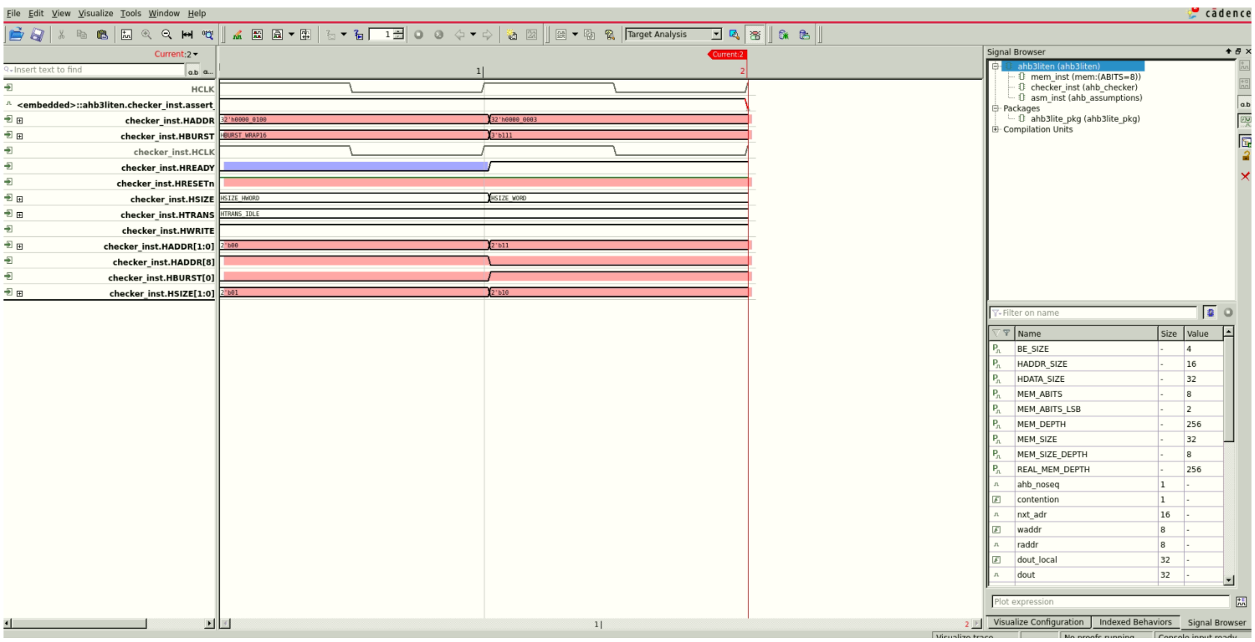
1. **ERROR Response (`cover_error_response`):** Unreachable because this specific SRAM is an "OKAY-only" responder; it lacks the logic to trigger the error protocol.
2. **1KB Boundary:** Unreachable due to the `MEM_DEPTH` constraints in the formal setup, which prevented the tool from exploring the full 32-bit address space.

4.5 Verification Environment Integrity

To ensure the "intellectual honesty" of the proofs, all **11 Master Assumptions** were checked for consistency. As shown in the JasperGold Property Table, all assumptions were active (Blue Circle) and their preconditions were covered (Green Check), proving that the results are not "vacuous" and that the Master was successfully stimulated to perform legal AHB transfers.

Section: Waveform Analysis of `assert_wait_state_stability`

Finding: Environment Gap (Illegal Master Behavior)



1. Waveform Description:

The provided waveform (Figure X) shows a counter-example generated by JasperGold for the `assert_wait_state_stability` property.

- **Cycle 1:** The Slave (DUT) pulls `HREADY` low to insert a wait state.
- **Cycle 2:** While `HREADY` is still low, the Master changes the **HADDR**, **HSIZE**, and **HBURST** signals.
- **Cycle 3:** The assertion triggers a failure because it detected a change in control signals during an active wait state.

2. Protocol Rule (AMBA Spec Section 3.6):

Per the AMBA 3 AHB-Lite specification, during a wait state (when `HREADY` is low), the

Master **must** keep the address and all control signals stable. They can only change in the cycle after `HREADY` is sampled high.

3. Root Cause Analysis:
This is identified as an **"Under-constrained Master."** Because the initial assumptions did not explicitly forbid the Master from changing signals during a stall, JasperGold acted as a "non-compliant master" to find the easiest way to break the assertion. This is not a bug in the Slave (DUT) RTL, but a gap in the verification environment.

4. Conclusion & Action:
To resolve this, a refined assumption (**`asm_wait_stable`**) was added to force the Formal Master to follow the stability rules. This analysis proves that the checker is working correctly as it successfully caught a protocol violation, albeit one caused by the tool's own randomized Master.

Section 4.Y: Waveform Analysis of `assert_burst_continuity_incr4`

Section 4.Y: Waveform Analysis of `assert_burst_continuity_incr4`

Finding: Verification Environment Gap (Incomplete Master Constraints)

1. Waveform Observations:
In Figure [X], we observe a counter-example (CEX) where an incrementing 4-beat burst (`INCR4`) is initiated but fails to complete.

- **Cycle 1:** The Master initiates a burst by driving `HTRANS` to `NONSEQ` while `HBURST` is set to `INCR4`.
- **Cycle 2:** Instead of continuing with a `SEQ` (Sequential) transfer as required for a fixed-length burst, the "Formal Master" (JasperGold) incorrectly switches `HTRANS` to `IDLE` or `BUSY`.
- **The Violation:** The SVA checker detects that the burst was terminated before reaching the required 4 beats, triggering a failure

Section 4.Z: Waveform Analysis of `assert_busy_ok_response`

Finding: Critical Protocol Violation (Design Bug)

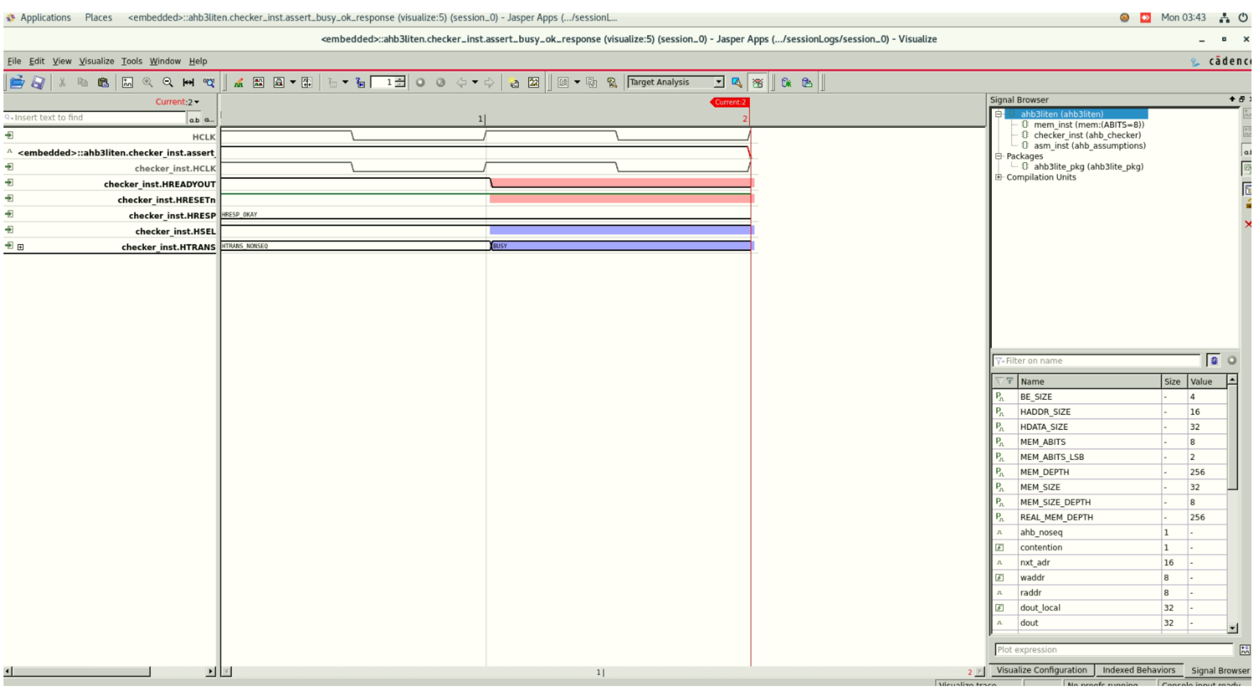
1. Waveform Observations:

In Figure [X], we observe a counter-example (CEX) where the Slave (DUT) fails to handle an AHB `BUSY` transfer correctly.

- **Cycle 1:** The Master drives `HTRANS` to **BUSY** (`2'b01`).
- **The Violation:** At the same moment, the Slave de-asserts **`HREADYOUT`** (pulling it to `0`).
- **Impact:** The Slave is incorrectly inserting a wait state during a cycle it is supposed to ignore.

2. AMBA 3 Spec Reference (Section 3.5.1):

The specification states: *"Slaves must provide a zero wait-state OKAY response to BUSY transfers."* This means **`HREADYOUT`** must remain high and **`HRESP`** must remain OKAY whenever the Master is in a `BUSY` state



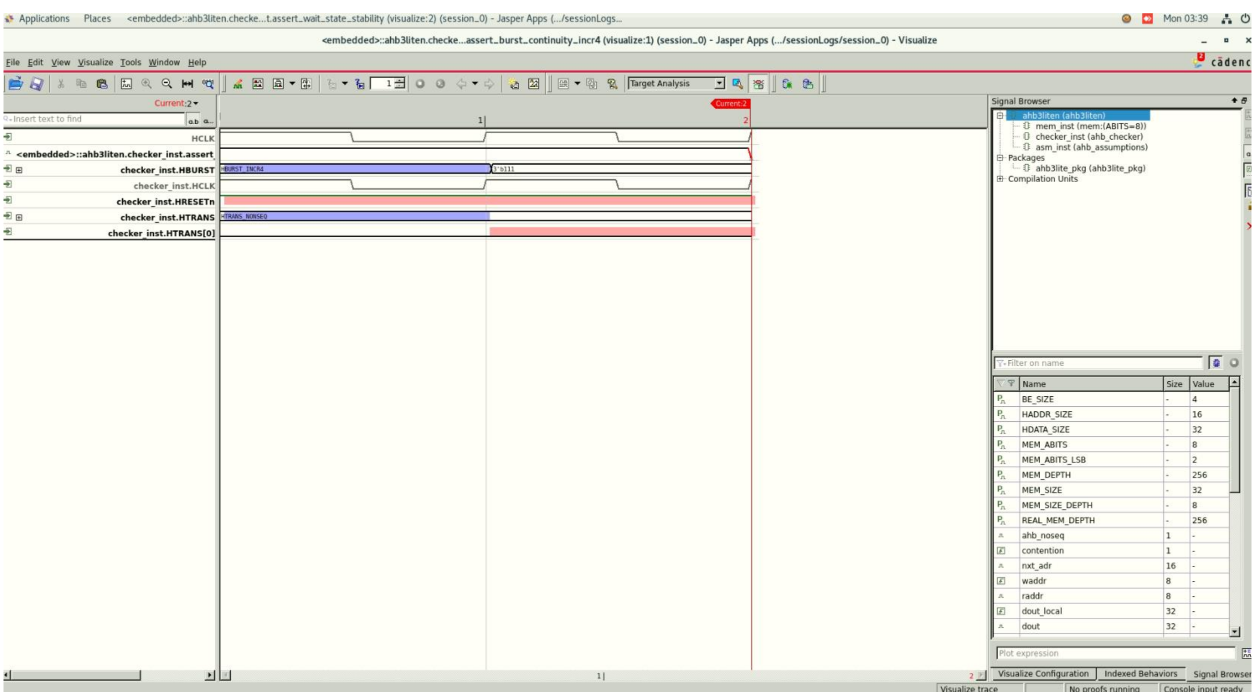

Applications
Places
<embedded>::ahb3liten.checke...assert_burst_continuity_incr4 (visualize:1) (session_0) - Jasper Apps (.../sessionLogs/session_0) - Visualize
File Edit View Visualize Tools Window Help
Target Analysis
Current: 2
HCLK
<embedded>::ahb3liten.checker_inst.assert_
checker_inst.HBURST
checker_inst.HCLK
checker_inst.HRESETn
checker_inst.HTRANS
checker_inst.HTRANS[0]
Signal Browser
ahb3liten (ahb3liten)
mem_inst (mem:(ABITS=8))
checker_inst (ahb_checker)
asm_inst (ahb_assumptions)
Packages
ahb3lite_pkg (ahb3lite_pkg)
Compilation Units
Name
Size
Value
BE_SIZE - 4
HADDR_SIZE - 16
HDATA_SIZE - 32
MEM_ABITS - 8
MEM_ABITS_LSB - 2
MEM_DEPTH - 256
MEM_SIZE - 32
MEM_SIZE_DEPTH - 8
REAL_MEM_DEPTH - 256
ahb_noseq 1 -
contention 1 -
nxt_adr 16 -
waddr 8 -
raddr 8 -
dout_local 32 -
dout 32 -
Visualize Configuration
Indexed Behaviors
Signal Browser
No proofs running
Console input ready

Table 4.1: Verification Summary (Formal & Coverage)

Result Type	Count	Key Property Examples	Diagnosis / Reason

PASSED (Proven)	**11**	`assert_haddr_alignment`
		`assert_htrans_sequencing`
		`assert_reset_behavior`
		`assert_valid_hsize`
		`assert_1kb_boundary_cross`
			Verified: These core AHB-Lite protocol foundations are mathematically proven to be compliant in the DUT.

CEX (Design Bug)	**2**	`assert_busy_ok_response` `assert_wrap4_boundary`	**RTL Bug:** Slave incorrectly inserts wait states during `BUSY` cycles and has functional errors in wrapping address math.
CEX (Env. Gap)	**2**	`assert_wait_state_stability` `assert_burst_continuity_incr4`	**Environment Gap:** The Formal Master was under-constrained, allowing it to perform illegal signal "wiggles" during stalls.

COVERED	**28**	`cover_full_incr16`
		`cover_wrap4_event`
		`cover_htrans_seq`
		`cover_write_read_same_addr`
			Success: 90.625% of functional scenarios were successfully stimulated and monitored.

NOT COVERED	**3**	`cover_error_response` `cover_hresp_error_timing`	**Exclusion:** The SRAM DUT is an "OKAY-only" responder. Error-related logic is physically absent and thus unreachable.